

TP d'Algorithmique
N°10

Table des matières :

Fonction afficher_bibliothèque :.....	2
Fonction afficher_liste_auteurs :.....	3
Fonction afficher_menu :.....	4
Fonction ajouter_auteur :.....	5
Fonction ajouter_livre :.....	7
Fonction rechercher_livre :.....	8
Programme principal :.....	10
Type Livre :.....	12
Type Auteur :.....	13
Liens entre les deux types :.....	14
Jeux d'essais :.....	14

Fonction afficher_bibliothèque :

```
def afficher_bibliothèque(bibliothèque: list[Livre]) -> None:
    """
    Fonction qui affiche l'ensemble des livres de la bibliothèque
    Args:
        bibliothèque (list[Livre]): la liste des livres de la bibliothèque
    Returns:
        None
    """
    for livre in bibliothèque:
        print(livre)
```

La fonction **afficher_bibliothèque** est conçue pour afficher tous les livres présents dans une bibliothèque, représentée sous forme de liste d'objets de type **Livre**.

Détails du code :

1. Docstring de la fonction :

- Explique :
 - Que la fonction prend en paramètre une liste de livres (**list[Livre]**).
 - Que la fonction affiche les livres et ne retourne rien (**None**).

2. Paramètre :

- **bibliothèque** : une liste d'objets de type **Livre**, où chaque livre contient des informations telles que le titre, l'auteur, l'année de publication, etc.

3. Traitement :

- **Boucle for** :
 - Parcourt chaque livre dans la liste **bibliothèque**.
 - Utilise la méthode **__str__** de la classe **Livre** (déjà définie) pour afficher les informations du livre.
- **Condition if** :
 - Vérifie si la liste **bibliothèque** est vide en utilisant **len(bibliothèque) == 0**.
 - Si c'est le cas, affiche : **"La bibliothèque est vide"**.

4. Type de retour :

- La fonction n'a pas de valeur de retour explicite. Elle affiche directement les informations.

Fonction afficher_liste_auteurs :

```
def afficher_liste_auteurs(ListeAuteur: list[Auteur]) -> None:
    """
    Fonction qui affiche l'ensemble des auteurs de la bibliothèque avec leurs indices
    Args:
        ListeAuteur (list[Auteur]): la liste des auteurs de la bibliothèque
    Returns:
        None
    """
    if not ListeAuteur:
        print("La liste des auteurs est vide")
        return

    print("Liste des auteurs :")
    for i, auteur in enumerate(ListeAuteur):
        print(f"{i} --> {auteur}")
```

Elle a exactement le même fonctionnement que la fonction *afficher_bibliothèque*. Elle se sert de l'affichage fourni dans la classe pour afficher tous les auteurs compris dans la liste.

Fonction afficher_menu :

```
def afficher_menu() -> int:
    """
    Fonction qui affiche le menu de la bibliothèque et qui retourne le choix de l'utilisateur
    Args:
        None : None
    Returns:
        int: le choix de l'utilisateur
    """
    choix : int

    print("1. Afficher l'ensemble des livres de la bibliothèque")
    print("2. Ajouter un nouveau livre")
    print("3. Ajouter un nouvel auteur")
    print("4. Rechercher un livre par son titre")
    print("5. Quitter")

    choix = int(input("Entrez votre choix: "))

    return choix
```

Cette fonction **afficher_menu** est utilisée pour afficher un menu d'options liées à une bibliothèque, puis récupérer et retourner le choix de l'utilisateur.

Détails du code :

1. **Docstring de la fonction :**
 - Elle explique :
 1. Que la fonction affiche un menu.
 2. Qu'elle retourne un entier correspondant au choix de l'utilisateur.
2. **Affichage du menu :**
 - Cinq options sont proposées :
 1. **Afficher l'ensemble des livres de la bibliothèque.**
 2. **Ajouter un nouveau livre.**
 3. **Ajouter un nouvel auteur**
 4. **Rechercher un livre par son titre.**
 5. **Quitter.**
3. **Saisie de l'utilisateur :**
 - La fonction demande à l'utilisateur d'entrer un choix en utilisant **input**.
 - La saisie est convertie en entier avec **int()**.
 - Ce choix est ensuite renvoyé par la fonction.
4. **Type de retour :**
 - Le type de retour est un entier (**int**) correspondant au numéro de l'option choisie.

Fonction ajouter_auteur :

```
def ajouter_auteur(listeAuteurs : list[Auteur]) -> None:
    """
    Fonction qui ajoute un auteur à la bibliothèque
    Args:
        bibliothèque (list[Livre]): la liste des livres de la bibliothèque
    Returns:
        None
    """

    nom : str
    nom = input("Entrez le nom de l'auteur: ")

    prenom : str
    prenom = input("Entrez le prénom de l'auteur: ")

    nationalite : str
    nationalite = input("Entrez la nationalité de l'auteur: ")

    dateNaissance : str
    dateNaissance = input("Entrez la date de naissance de l'auteur: ")

    choix = input("Est-il décédé ?").capitalize()
    if choix == "Oui" :
        datedeces : str
        datedeces = input("Quelle est sa date de décès :")
    else :
        print("Alors il est vivant")
        datedeces = ""

    auteur = Auteur(nom, prenom, nationalite, dateNaissance, datedeces)
    listeAuteurs.append(auteur)
```

La fonction *ajouter_auteur* permet d'ajouter un nouvel auteur à une liste d'auteurs représentée par une liste d'objets de type *Auteur*.

Détails du code :

Docstring de la fonction :

Explique que :

- La fonction prend en entrée une liste d'auteurs.
- Elle ne retourne rien (None) mais modifie la liste en y ajoutant un nouvel auteur.

Paramètre :

- `listeAuteurs` : une liste où chaque élément est un objet Auteur.

Traitement :

Demande d'informations à l'utilisateur :

La fonction interagit avec l'utilisateur pour obtenir les détails de l'auteur :

- **Nom** : récupéré comme une chaîne de caractères (str).
- **Prénom** : récupéré comme une chaîne de caractères (str).
- **Nationalité** : récupérée comme une chaîne de caractères (str).
- **Date de naissance** : récupérée comme une chaîne de caractères (str).
- **Date de décès** :
 - Si l'utilisateur répond "Oui" à la question "Est-il décédé ?", la date de décès est demandée (chaîne de caractères).
 - Sinon, l'auteur est considéré vivant, et la date de décès est laissée vide ("").

Création d'un objet Auteur :

Un objet Auteur est créé en utilisant les informations saisies.

Ajout à la liste d'auteurs :

L'objet Auteur est ajouté à la liste *listeAuteurs* grâce à la méthode `append`.

Type de retour :

La fonction ne retourne pas de valeur, elle modifie directement la liste passée en argument.

Fonction ajouter livre :

```
def ajouter_livre(bibliothèque: list[Livre], listeAuteurs : list[Auteur]) -> None:
    """
    Fonction qui ajoute un livre à la bibliothèque
    Args:
        bibliothèque (list[Livre]): la liste des livres de la bibliothèque
    Returns:
        None
    """
    titre : str
    titre = input("Entrez le titre du livre: ")

    auteur : Auteur
    afficher_liste_auteurs(listeAuteurs)
    choixauteur = int(input("Choisissez l'auteur que vous souhaitez (Entrez le chiffre noté sur la gauche) : "))
    auteur = listeAuteurs[choixauteur]

    annee : int
    annee = int(input("Entrez l'année de publication: "))

    nbpages : int
    nbpages = int(input("Entrez le nombre de pages: "))

    livre = Livre(titre, auteur, annee, nbpages)
    bibliothèque.append(livre)
```

La fonction **ajouter_livre** permet d'ajouter un nouveau livre à une bibliothèque représentée par une liste d'objets de type **Livre**.

Détails du code :

1. Docstring de la fonction :

- Explique que :
 - La fonction prend en entrée une liste de livres (**bibliothèque**).
 - Elle ne retourne rien (**None**) mais modifie la liste en y ajoutant un nouveau livre.

2. Paramètre :

- **bibliothèque** : une liste où chaque élément est un objet **Livre**.

3. Traitement :

- **Demande d'informations à l'utilisateur :**
 - La fonction interagit avec l'utilisateur pour obtenir les détails du livre :
 - **Titre** : récupéré comme une chaîne de caractères.
 - **Auteur** : récupéré par le choix de l'utilisateur dans la listeAuteur
 - **Année de publication** : récupérée comme un entier.
 - **Nombre de pages** : récupéré comme un entier.
- **Création d'un objet Livre :**
 - Utilise les informations saisies pour créer un nouvel objet de type **Livre**.
- **Ajout à la bibliothèque :**
 - L'objet **Livre** est ajouté à la liste **bibliothèque** grâce à la méthode **append**.

4. Type de retour :

- La fonction ne retourne pas de valeur, elle modifie directement la liste passée en argument.

Fonction rechercher_livre :

```
def rechercher_livre(bibliothèque: list[Livre]) -> None:
    """
    Fonction qui recherche un livre par son titre
    Args:
        bibliothèque (list[Livre]): la liste des livres de la bibliothèque
    Returns:
        None
    """
    titre = input("Entrez le titre du livre: ")
    trouve = False # Indicateur pour savoir si le livre a été trouvé

    for livre in bibliothèque:
        if livre.titre == titre:
            print(livre)
            trouve = True

    if not trouve: # Si aucun livre n'a été trouvé
        print("Le livre n'existe pas dans la bibliothèque")
```

La fonction **rechercher_livre** permet de chercher un livre dans une bibliothèque (représentée par une liste d'objets de type **Livre**) en se basant sur le **titre** fourni par l'utilisateur. Elle affiche

les informations du livre trouvé ou informe que le livre n'existe pas si aucun résultat ne correspond

1. Demande du titre :

- La fonction invite l'utilisateur à entrer le titre du livre qu'il cherche.

2. Parcours de la bibliothèque :

- La fonction utilise une boucle **for** pour examiner chaque livre dans la liste.

3. Vérification :

- Si un livre à un titre qui correspond exactement à celui saisi (**livre.titre == titre**), la fonction affiche les informations du livre en utilisant la méthode **__str__** de l'objet **Livre**.
- Une variable **trouve** est utilisée pour indiquer qu'un livre a été trouvé.

4. Message d'erreur (si nécessaire) :

- Une fois la boucle terminée, si **trouve** est toujours **False** (aucun livre trouvé), la fonction affiche le message : **"Le livre n'existe pas dans la bibliothèque"**.

5. Type de retour :

- La fonction ne retourne rien (**None**), elle affiche uniquement des messages.

Programme principal :

```
if __name__ == "__main__":
    bibliothèque : list[Livre] = []
    ListeAuteurs : list[Auteur]
    ListeAuteurs = [
        Auteur("Hugo", "Victor", "Française", "26/02/1802", "22/05/1885"),
        Auteur("Austen", "Jane", "Anglaise", "16/12/1775", "18/07/1817"),
        Auteur("Tolstoï", "Léon", "Russe", "09/09/1828", "20/11/1910"),
        Auteur("Rowling", "J.K.", "Britannique", "31/07/1965", ""), # Auteur vivant
        Auteur("Murakami", "Haruki", "Japonais", "12/01/1949", ""), # Auteur vivant
        Auteur("Hemingway", "Ernest", "Américaine", "21/07/1899", "02/07/1961"),
        Auteur("Coelho", "Paulo", "Brésilienne", "24/08/1947", ""), # Auteur vivant
        Auteur("Camus", "Albert", "Française", "07/11/1913", "04/01/1960"),
        Auteur("Dumas", "Alexandre", "Française", "24/07/1802", "05/12/1870"),
        Auteur("Gaiman", "Neil", "Britannique", "10/11/1960", "") # Auteur vivant
    ]

    choix : int

    while True:
        choix = afficher_menu()

        if choix == 1:
            afficher_bibliothèque(bibliothèque)
        if choix == 2:
            ajouter_livre(bibliothèque, ListeAuteurs)
        if choix == 3:
            ajouter_auteur(ListeAuteurs)
        if choix == 4:
            rechercher_livre(bibliothèque)
        if choix == 5:
            exit()
        if choix < 1 or choix > 5:
            print("Choix invalide")
```

Ce code est le **point d'entrée principal** du programme, qui combine les différentes fonctionnalités pour gérer une bibliothèque de **livres** et **auteurs** via un menu interactif.

1. Initialisation

a) Liste des livres (bibliothèque)

- Une liste vide est déclarée pour stocker les livres.
- Elle représente la bibliothèque que l'utilisateur peut remplir au cours de l'exécution.

b) Liste des auteurs (ListeAuteurs)

- Une liste prédéfinie d'auteurs célèbres est initialisée. Chaque auteur est instancié avec ses informations : nom, prénom, nationalité, date de naissance, et éventuellement date de décès (vide pour les auteurs vivants).

c) Variable de choix (choix)

- Une variable est déclarée pour enregistrer le choix de l'utilisateur dans le menu interactif.

2. Boucle principale

- Une boucle infinie (while True) permet au programme de rester actif tant que l'utilisateur ne choisit pas de le quitter.
- À chaque itération, le menu est affiché et l'utilisateur est invité à faire un choix.

3. Affichage du menu

- Le programme affiche un menu interactif via la fonction `afficher_menu`, qui retourne le choix de l'utilisateur.
- Les différentes options proposées permettent d'interagir avec les livres et les auteurs.

4. Traitement des choix

Choix possibles :

1. **Afficher la bibliothèque :**
Affiche tous les livres présents dans la bibliothèque.
2. **Ajouter un livre :**
Permet à l'utilisateur d'ajouter un nouveau livre à la bibliothèque en saisissant ses informations.
La liste des auteurs est passée en paramètre pour associer un auteur existant.
3. **Ajouter un auteur :**
Permet d'ajouter un nouvel auteur à la liste des auteurs existants.

4. **Rechercher un livre :**

Permet de rechercher un livre dans la bibliothèque à partir de son titre.

5. **Quitter :**

Termine l'exécution du programme.

Gestion des choix invalides :

Si l'utilisateur saisit une option hors du cadre défini (par exemple, < 1 ou > 5), un message d'erreur "Choix invalide" est affiché.

Type Livre :

```
class Livre:
    """
    Classe qui représente un livre
    Attributs:
        titre (str): le titre du livre
        auteur (str): le nom de l'auteur
        annee (int): l'année de publication
        nbPages (int): le nombre de pages
    """

    def __init__(self, titre: str, auteur: str, annee: int, nbpages: int):
        self.titre = titre
        self.auteur = auteur
        self.annee = annee
        self.nbPages = nbpages

    def __str__(self):
        return f"Titre: {self.titre}, Auteur: {self.auteur}, Année de publication: {self.annee}, Nombre de pages: {self.nbPages}"
```

Ce code définit une classe Python appelée **Livre**, qui permet de représenter un livre avec ses principales caractéristiques.

Détails du code :

1. **Docstring de la classe :**

- Une description de la classe et de ses attributs :
 - titre : le titre du livre (chaîne de caractères).
 - auteur : le nom de l'auteur (chaîne de caractères).
 - annee : l'année où le livre a été publié (entier).
 - nbPages : le nombre de pages du livre (entier).

2. **Constructeur (__init__) :**

- Une méthode spéciale utilisée pour créer un objet de type **Livre**.
- Elle prend quatre paramètres :
 - **titre, auteur, annee, et nbpages.**

- Ces paramètres sont utilisés pour initialiser les attributs de l'objet.

3. Méthode spéciale `__str__` :

- Permet de définir comment afficher un objet **Livre** sous forme de texte.
- Retourne une chaîne de caractères formatée contenant les informations du livre (titre, auteur, année, nombre de pages).

Type Auteur :

```
class Auteur :
    """
    Classe qui représente un auteur
    Args:
        nom (str): le nom de l'auteur
        prenom (str): le prénom de l'auteur
        dateNaissance (str): la date de naissance de l'auteur
        nationalite (str): la nationalité de l'auteur
        DateDeces (str): la date de décès de l'auteur
    """

    def __init__(self, nom: str, prenom: str, nationalite: str, dateNaissance: str, dateDeces: str) :
        self.nom = nom
        self.prenom = prenom
        self.nationalite = nationalite
        self.dateNaissance = dateNaissance
        self.DateDeces = None # Par défaut, l'auteur est en vie

    def __str__(self):
        return f"Nom : {self.nom} | Prénom : {self.prenom} | Nationalité : {self.nationalite} | Date de naissance : {self.dateNaissance} | Date de décès : {self.DateDeces}"
```

Cette nommée **Auteur**, qui sert à représenter les informations relatives à un auteur, comme son nom, son prénom, sa nationalité, sa date de naissance et, éventuellement, sa date de décès.

Voici une description simple des éléments du code :

1. Classe **Auteur** :

Une structure qui permet de stocker des informations sur un auteur.

2. Attributs de la classe :

- *nom* : Le nom de l'auteur.
- *prenom* : Le prénom de l'auteur.
- *nationalite* : La nationalité de l'auteur.
- *dateNaissance* : La date de naissance de l'auteur.
- *DateDeces* : La date de décès de l'auteur (par défaut None, ce qui signifie que l'auteur est supposé être en vie).

3. Méthode `__init__` :

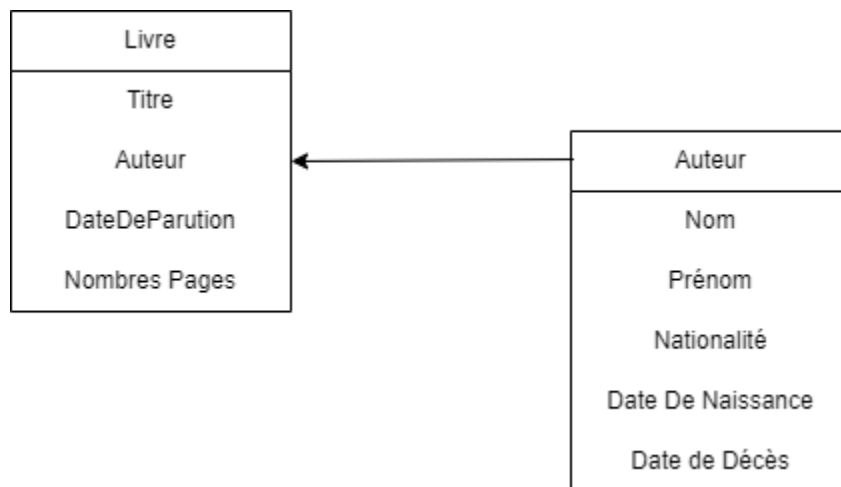
C'est le constructeur de la classe. Il initialise les attributs avec les valeurs passées en

arguments lors de la création d'un objet Auteur. Si la date de décès n'est pas précisée, elle est automatiquement définie à None.

4. **Méthode `__str__` :**

Une méthode spéciale qui permet de représenter l'objet Auteur sous forme de texte lisible. Par exemple, quand on affiche un objet Auteur avec `print`, cette méthode retourne une chaîne contenant toutes les informations de l'auteur.

Liens entre les deux types :



Jeux d'essais :

```
1. Afficher l'ensembles des livres de la bibliothèque
2. Ajouter un nouveau livre
3. Rechercher un livre par son titre
4. Quitter
Entrez votre choix: 
```

Au lancement du programme , on affiche le menu afin d'indiquer à l'utilisateur les choix qui lui sont possibles.

```
Entrez votre choix: 7
Choix invalide
1. Afficher l'ensembles des livres de la bibliothèque
2. Ajouter un nouveau livre
3. Rechercher un livre par son titre
4. Quitter
Entrez votre choix: █
```

Si l'utilisateur ne rentre pas un choix valide, cela le ramène sur le menu en lui indiquant qu'il a fait un mauvais choix.

```
Entrez votre choix: 1
La bibliothèque est vide
```

Si l'utilisateur veut afficher la bibliothèque mais qu'elle est vide, cela lui est indiqué.

```
Entrez votre choix: 2
Entrez le titre du livre: Mamamia
Entrez le nom de l'auteur: DIDRY
Entrez l'année de publication: 2019
Entrez le nombre de pages: 2
```

Quand il veut entrer un nouveau livre, le programme lui demande la saisie de tous les paramètres de la classe Livre.

```
Titre: moi, Auteur: moche, Année de publication: 2014, Nombre de pages: 25
Titre: oele, Auteur: æf, Année de publication: 2688, Nombre de pages: 45
```

Si il ré-affiche la bibliothèque après y avoir inséré des nouveaux livres, le programme les lui affichera dans l'ordre de saisie.

```
Entrez votre choix: 3
Entrez le titre du livre: moi
Titre: moi, Auteur: moche, Année de publication: 2014, Nombre de pages: 25
```

```
Entrez votre choix: 3
Entrez le titre du livre: Les misérables
La bibliothèque ne comporte pas votre livre
```

Enfin la recherche de livre permet à l'utilisateur de rentrer le titre d'un livre ce qui va lui renvoyer soit toutes les informations du livre s'il est présent, sinon un message d'erreur.

```
4. Quitter
Entrez votre choix: 4
11:02 ducouret17@iutinfo-105-07 Programme TP $ █
```

Pour finir, si l'utilisateur souhaite quitter le programme, il le peut en sélectionnant le choix 4.